

Согласно реляционной модели Кодда, система управления базами данных должна включать поддержку языка, обеспечивающего манипулирование структурами БД и содержащейся в этих структурах информацией.

Стандартом де-факто такого языка на текущий момент является **Structured Query Language - SQL**.

Существует ANSI стандарт этого языка, обычно поддерживаемый всеми производителями СУБД. Кроме того, часто разработчики ПО добавляют собственные расширения языка, которые, с одной стороны, призваны расширить доступный пользователю функционал, а, с другой, косвенно привязать пользователя к конкретным продуктам.

Мы будем говорить о диалекте **Transact SQL**, поддерживаемом СУБД **Sybase ASE**.

1. Объекты базы данных

В базе данных содержатся различные объекты: таблицы, сохраненные процедуры, пользовательские функции, триггеры, представления, правила и т.д.

Каждый объект характеризуется своим именем. Полное имя для большинства объектов, в том числе для таблиц, формируется как:

```
имя_базы.имя_владельца.имя_объекта
```

Таким образом, если в базе **db1** есть таблица **someTable**, принадлежащая пользователю **user1**, ее полное имя будет

```
db1.user1.someTable
```

Если речь идет об объекте в текущей базе данных или объекте, принадлежащем текущему пользователю (или владельцу базы данных - **dbo**), соответствующий элемент имени можно пропускать:

db1..someTable	таблица в базе db1 , принадлежащая текущему пользователю или dbo
user1.someTable	таблица в текущей базе, принадлежащая пользователю user1
someTable	таблица в текущей базе, принадлежащая текущему пользователю или dbo

Для обращения к объектам, принадлежащим **dbo**, у текущего пользователя должны быть соответствующие права.

Таблицы в ASE можно разделить на два типа: **пользовательские** и **системные**.

Системные таблицы, часто называемые, "**data dictionary**" или "**системный каталог**", поддерживаются самим сервером и содержат информацию обо всех объектах базы данных, включая саму базу данных, пользовательских и системных таблицах, учетных записях пользователей.

Имена таблиц для Sybase ASE должны удовлетворять следующим правилам:

- максимальная длина имени таблицы - **30** символов
- первый символ имени должен быть символом латинского алфавита, символом подчеркивания (**_**) или диэзом (**#**)
- остальные символы имени таблицы должны принадлежать набору **[A-Z, a-z, 0-9, _, \$, #]**
- имя таблицы не должно быть зарезервированным словом Sybase ASE
- имя объекта должно быть уникальным для владельца

Кроме того, при именовании и использовании таблиц и других объектов БД, следует помнить о следующих рекомендациях и особенностях:

- старайтесь давать объектам СУБД осмысленные названия
- чувствительность к регистру зависит от установленного на сервере порядка сортировки (**sort order**)

2. Поля таблиц

Каждая таблица состоит из полей. Поле характеризуется своим именем. Также с каждым полем ассоциирован тип данных, хранимых в этом поле.

Правила именования полей таблиц:

- максимальная длина имени поля - **30** символов
- первый символ имени должен быть символом латинского алфавита или символом подчеркивания (**_**)
- остальные символы имени поля должны принадлежать набору **[A-Z, a-z, 0-9, _]**
- имя поля может быть зарезервированным словом Sybase ASE, но использовать такие имена не рекомендуется, для обращения к ним потребуется обязательно заключать имя поля в кавычки, это потенциально может стать источником проблем
- имя поля должно быть уникальным в рамках таблицы

Для полей справедливы рекомендации по именованию любых объектов БД, кроме того, рекомендуется полям разных таблиц, содержащим одни и те же сущности, давать одинаковые имена.

3. Типы данных

Основные типы типов данных приведены ниже.

char(n) varchar(n)	Строки с максимальной длиной n символов. Поля с типом char дополняются пробелами до длины n .
nchar(n) nvarchar(n)	Unicode-аналоги типов char(n) и varchar(n)
binary(n) varbinary(n)	Бинарные данные в 16-ричной нотации. Данные начинаются с “0x” и состоят из символов [0-9, A-F] . В остальном эти типы аналогичны char(n) и varchar(n)
tinyint	целое число в диапазоне от 0 до 255 (1 байт)
smallint	целое число от -32768 до +32767 (2 байта)
int	целое число от -2147483648 до +2147483647 (4 байта)
smalldatetime	дата и время с 1.01.1900 по 6.06.2079 с точностью до 1 минуты
datetime	дата и время с 1.01.1753 по 31.12.9999 с точностью до 1/300 секунды
text image	крупные текстовые (text) или бинарные (image) блоки размером от 1 страницы сервера до 2GB
numeric(p,s) decimal(p,s)	Числа с дробной частью. P - размерность числа (общее количество разрядов), s - точность представления (количество знаков после запятой). Если s = 0 , этот параметр можно не указывать. Возможный размер - от 2 до 17 байт, допустимый диапазон значений - от -10³⁸ до (+10³⁸-1) . Типы numeric и decimal полностью идентичны, за исключением одного: только numeric с точностью 0 можно использовать в качестве полей identity .
bit	Аналог булевого (логического) типа. Содержит 0 или 1 . Допускается ввод других целых чисел, но все они интерпретируются как 1 . Столбцы типа bit не допускают значения NULL и по ним нельзя построить индекс.
timestamp	При любом изменении строки, содержащей столбец типа timestamp , его значение автоматически модифицируется. Таблица может иметь только один столбец timestamp . Столбцу с именем “ timestamp ” автоматически присваивается тип timestamp . Тип timestamp определен как varbinary(16) null .
sysname	Специальный тип данных, который используется в системных таблицах. Определен как varchar(30) null . Недоступен для использования в пользовательских таблицах.

Данный список типов данных не претендует на полноту, с полным списком можно ознакомиться в документации по Sybase ASE.

4. Константы SQL

Для записи констант различных типов применяются следующие правила.

Числовые константы.

Десятичные	Цифры от 0 до 9. В качестве разделителя целой и дробной части используется точка (.). Для отрицательных чисел в начале константы указывается символ “-” (минус). Для положительных чисел можно перед числом указывать символ “+” (плюс). Примеры: 25 13.64 -528 +12.33 Также допускается запись в формате <число>e<степень 10> Например, 2.5e3 = 2500 (2.5 умножить на 10³)
Шестнадцатеричные	Константа начинается с символов “0x”, после чего идут символы [0-9, A-F, a-f], при помощи которых записывается шестнадцатеричное значение числа. Пример: 0x22 = 34 (22 шестнадцатеричное - это 34 десятичное)
Восьмеричные	Константа начинается с символа “0”, после чего идут символы [0-7], при помощи которых записывается восьмеричное значение числа. Пример: 022 = 18 (22 восьмеричное - это 18 десятичное)

Строковые константы.

Строковые константы заключаются в одинарные кавычки (апострофы - <'>). Если апостроф нужно включить в значение константы, он повторяется дважды.

Примеры:

```
'Иванов Иван Иванович'  
'Контора ''Рога и копыта'''
```

Дата и время

Константы дат записываются аналогично строкам - заключенными в апострофы. Sybase ASE автоматически распознает следующие форматы дат и времени:

- YYYY-MM-DD [hh:mm[:ss[.sss]] [AM/PM]]
- YY-MM-DD [hh:mm[:ss[.sss]] [AM/PM]]
- YYYYMMDD [hh:mm[:ss[.sss]] [AM/PM]]
- YYMMDD [hh:mm[:ss[.sss]] [AM/PM]]
- Mon DD YYYY [hh:mm[:ss[.sss]] [AM/PM]]

Примеры:

```
'2012-04-22'
'20120422 18:32'
'2012-04-22 18:32:11.54'
'2012-04-22 6:32:11 PM'
'Apr 22 2012 6:32 PM'
```

5. Значения NULL

Зачастую значение того или иного атрибута объекта (поля таблицы) неизвестно. Для таких случаев практически все СУБД поддерживают значение **NULL**.

NULL не является аналогом пустой строки или арифметического нуля. **NULL** говорит о том, что значение поля **не известно**.

Для того, чтобы конкретное поле могло содержать значения **NULL**, оно должно быть соответствующим образом описано. Этот вопрос будет позднее рассмотрен более подробно.

6. Синтаксис SQL. Основные соглашения.

Основные правила и рекомендации по написанию SQL-выражений:

- операторы SQL нечувствительны к регистру символов
- операторы SQL могут записываться в несколько строк
- отдельные ключевые слова записываются в одну строку
- порядок ключевых слов в предложении должен строго соблюдаться
- сокращения ключевых слов не допускаются
- предложения SQL обычно записываются с новой строки
- для лучшей читаемости при написании запросов обычно используют отступы и табуляцию

7. Комментарии.

В тексты SQL-запросов можно добавлять комментарии.

Комментарии бывают двух типов:

- **многострочные** - начало комментария обозначается символами `/*`, окончание - `*/`
- **однострочные** - начало комментария обозначается символами `--`, комментарий продолжается до конца строки

Примеры:

```
/* Это  
многострочный  
комментарий */
```

```
-- Это однострочный комментарий
```

8. Создание таблиц: базовые сведения.

Подробно процесс создания таблиц будет рассмотрен позднее. Сейчас приведем базовый синтаксис, необходимый для понимания дальнейшего материала.

Создание таблицы выполняется при помощи инструкции

```
CREATE TABLE tableName ( columnName dataType [{null | not null}] [, ...] )
```

Здесь

- **tableName** - имя таблицы
- **columnName** - имя поля
- **dataType** - тип данных поля
- **null / not null** - признак того, что поле допускает или не допускает значения **NULL**

Пример:

```
CREATE TABLE People (  
    PeopleId int not null,  
    FirstName varchar(30) not null,  
    LastName varchar(70) not null,  
    DateBorn date null  
)
```

9. Оператор **SELECT**. Базовый синтаксис.

Оператор **SELECT** позволяет выполнять три основные операции по получению информации из таблиц базы данных:

- **проектирование** - определение списка выбираемых полей
- **выбор** - определение списка выбираемых строк
- **объединение** - выбор из нескольких таблиц с описанием взаимосвязей между ними

Базовый синтаксис оператора **SELECT** имеет следующий вид:

```
SELECT { ([tableName.]{* | columnName}) | expr } [, ...]  
FROM tableName [, ...]
```

Предложение **SELECT** содержит информацию о том, какие поля должны быть включены в выборку (resultset).

Элементы предложения **SELECT**:

- ***** - указание необходимости выбора всех полей
- **columnName** - имя поля таблицы
- **expr** - любое выражение, включающее арифметические выражения и/или вызовы встроенных функций
- **tableName** - имя таблицы, которое можно указывать перед именем поля, выбираемого из данной таблицы или символом “*”. В последнем варианте запись означает “Все поля из таблицы tableName”. В качестве разделителя имени таблицы и имени поля (или *) используется **точка**. Указание имени таблицы является обязательным, если поле с именем columnName присутствует более чем в одной из таблиц, перечисленных в предложении **FROM**.

Столбцы resultset, описываемые в выражении **SELECT**, разделяются запятыми. В разделе **FROM** перечисляются таблицы, из которых будет вестись выборка.

В вырожденном варианте предложение **FROM** может отсутствовать. В этом случае resultset будет содержать одну строку, а список полей формируется из выражений, содержащих только константы или переменные.

Примеры:

- Выбор всех данных из таблицы **myTable**:

```
SELECT * FROM myTable  
SELECT myTable.* FROM myTable
```
- Выбор указанных столбцов из таблицы **syslogins** базы **master**:

```
SELECT master..syslogins.suid, name, fullname FROM master..syslogins
```
- Формирование resultset из одной строки при помощи констант:

```
SELECT 25, 'Иванов Иван Иванович'
```

10. Использование псевдонимов.

При использовании оператора **SELECT** каждому полю resultset и каждой таблице в предложении **FROM** можно присвоить альтернативное имя - псевдоним.

Существует два варианта описания псевдонимов полей:

```
SELECT alias = { columnName | expr }
```

и

```
SELECT { columnName | expr } [as] alias
```

Если псевдоним столбца содержит пробелы или символы локальных кодировок, он заключается в двойные кавычки (").

Псевдонимы таблиц описываются одним способом:

```
FROM tableName [as] alias
```

Псевдонимы таблиц могут использоваться для уточнения имен полей в запросе.

Пример:

```
SELECT l.suid "ID пользователя",  
       l.name as Login,  
       l.fullname as "Полное имя"  
FROM master..syslogins l
```

11. Выражения.

Кроме полей таблиц в resultset можно включать выражения, результат которых вычисляется в процессе выполнения запроса.

Для числовых величин (полей таблиц или констант) можно применять следующие операторы:

+	Сложение
-	Вычитание
*	Умножение
/	Деление
%	Остаток от деления

Операторы имеют традиционный приоритет выполнения: сначала умножение и деление, потом сложение и вычитание. Для явного указания порядка выполнения операторов можно использовать скобки.

Пример:

```
SELECT Name, 12 * Salary + 300 as YearSalaryWithBonus
FROM People
```

Если в арифметическом выражении встречаются значения **NULL**, то результатом такого выражения всегда будет **NULL**:

```
5 + NULL = NULL
12 / NULL = NULL
```

Как уже говорилось ранее, **NULL** - это "неизвестно, что". Если к "неизвестно, чему" прибавить любое число, в результате все равно получится "неизвестно, что".

В выражениях также могут участвовать и строковые величины. К строкам применяется операция конкатенации (+):

```
SELECT 'Уважаемый(ая) ' + Name + '!' as LetterPrefix
FROM People
```

В отличие от арифметических выражений, при конкатенации **NULL** трактуется как пустая строка:

```
'Префикс ' + NULL + ' суффикс' = 'Префикс  суффикс'
```

12. Удаление дубликатов строк.

Оператор **SELECT** позволяет удалять дубликаты строк из resultset. Для этого служит ключевое слово **DISTINCT**, указываемое сразу после **SELECT**.

Пример:

```
SELECT LastName, FirstName from People
```

LastName	FirstName
Иванов	Иван
Сидоров	Сидор
Иванов	Иван
Сидоров	Сидор

```
SELECT DISTINCT LastName, FirstName from People
```

LastName	FirstName
Иванов	Иван
Сидоров	Сидор

По аналогии с **DISTINCT** используется ключевое слово **ALL**, которое отключает удаление дублирующихся строк. Однако на практике это ключевое слово не используется, поскольку без него оператор **SELECT** выполняется точно так же.

```
SELECT ALL LastName, FirstName from People
```

13. Объединение выборок.

Существует возможность объединить несколько выборок в один resultset. Для этого в каждой из объединяемых выборок должно быть одно и то же количество полей, кроме того, типы полей, находящиеся на соответствующих позициях, должны быть совместимы.

Для объединения выборок существуют ключевые слова UNION и UNION ALL, при помощи которых объединяются несколько операторов SELECT:

```
SELECT LastName, FirstName from Clients
```

LastName	FirstName
Иванов	Иван
Сидоров	Сидор

```
SELECT Name1, Name2 from Employees
```

Name1	Name2
Иванов	Иван
Петров	Петр

```
SELECT LastName, FirstName from Clients
```

```
UNION
```

```
SELECT Name1, Name2 from Employees
```

LastName	FirstName
Иванов	Иван
Сидоров	Сидор
Петров	Петр

Как видно из примера, в результирующей выборке удалены дублирующиеся строки (Иванов Иван присутствует как в таблице Clients, так и в Employees).

Если дубликаты удалять не нужно, используется инструкция **UNION ALL**:

```
SELECT LastName, FirstName from Clients
```

```
UNION ALL
```

```
SELECT Name1, Name2 from Employees
```

LastName	FirstName
Иванов	Иван
Сидоров	Сидор
Иванов	Иван
Петров	Петр

Так же **UNION ALL** рекомендуется использовать, если объединяемые выборки принципиально не могут содержать дублирующихся записей, поскольку обработка **UNION ALL** менее ресурсоемка:

```
SELECT 1 as Source, LastName, FirstName from Clients
UNION ALL
SELECT 2, Name1, Name2 from Employees
```

Source	LastName	FirstName
1	Иванов	Иван
1	Сидоров	Сидор
2	Иванов	Иван
2	Петров	Петр

14. Сортировка данных.

При выборе информации из базы данных строки `resultset` могут быть отсортированы в произвольном порядке. При необходимости получения выборки, отсортированной по значению определенных полей, это нужно указывать явно.

Для указания параметров сортировки в конце оператора **SELECT** может быть использовано предложение **ORDER BY**.

```
SELECT [ALL | DISTINCT] select_list
[from_clause]
[...]
[order_by_clause]
```

Здесь **order_by_clause**:

```
ORDER BY {columnName | index | expr} [ASC | DESC] [, ...]
```

- **columnName** - имя (или псевдоним) поля
- **index** - номер поля в `resultset`, начиная с 1
- **expr** - выражение
- **ASC / DESC** - ключевые слова, говорящие о том, что данные соответствующего поля должны сортироваться по возрастанию (ASC) или по убыванию (DESC). По умолчанию обычно принимается ASC, хотя для Sybase ASE это настраиваемый параметр.

Порядок сортировки зависит от используемого сервером набора символов (**charset**) и порядка сортировки (**sort order**)

Примеры.

```
-- Сортировка клиентов по фамилии в порядке убывания (от Я до А)
SELECT LastName, FirstName from Clients
ORDER BY LastName DESC
```

```
-- То же самое с использованием индекса поля
SELECT LastName, FirstName from Clients ORDER BY 1 DESC
-- Сортировка по выражению
SELECT Name, 12 * Salary + 300
FROM People
ORDER BY 12 * Salary + 300
```

```
-- То же самое с использованием псевдонима
SELECT Name, 12 * Salary + 300 as YearSalaryWithBonus
FROM People
ORDER BY YearSalaryWithBonus
```

Сортировка может производиться по нескольким полям. В этом случае порядок сортировки зависит от порядка перечисления полей в **ORDER BY**.

В данном примере сортировка идет сначала по фамилии, а при совпадении фамилий - по имени.

```
SELECT LastName, FirstName from Clients  
ORDER BY LastName, FirstName
```

LastName	FirstName
Петров	Иван
Петров	Сергей
Яковлев	Иван

А здесь - наоборот.

```
SELECT LastName, FirstName from Clients  
ORDER BY FirstName, LastName
```

LastName	FirstName
Петров	Иван
Яковлев	Иван
Петров	Сергей

Несмотря на то, что сравнение значений **NULL** - некорректная операция, для определенности принято, что, если в сортируемых полях содержатся значения **NULL**, они считаются "минимальными". Т.е., при сортировке по возрастанию значения **NULL** располагаются в начале resultset, а при сортировке по убыванию - в конце.

15. Ограничение размера ResultSet.

Достаточно часто возникает ситуация, когда нужно получить не весь ResultSet, сформированный в результате выполнения оператора SELECT, а, например, первые 10 или 100 записей. Данная задача решается двумя способами:

1. Установка параметра **ROWCOUNT**.

```
SET ROWCOUNT maxNumberOfRows
```

В результате выполнения приведенной выше инструкции количество строк, возвращаемых оператором SELECT, будет ограничено указанным максимальным значением. Это ограничение действует и на инструкции DML. Ограничение действует до момента закрытия пользовательской сессии или до установки нового ограничения. Если указать **maxNumberOfRows** равным нулю, ограничения снимаются.

2. Использование ключевого слова **TOP**.

В последних версиях ASE появилась поддержка в составе оператора SELECT ключевого слова TOP, которое позволяет ограничить количество строк, возвращаемых конкретным оператором.

Пример.

```
-- Выбор первых десяти записей из master..sysprocesses  
SELECT TOP 10 * FROM master..sysprocesses
```

Использование ограничений количества выводимых строк позволяет существенно ускорить выполнение запросов, если для анализа достаточно неполного объема данных. Однако нужно учитывать, что при использовании ORDER BY, ускорение будет не столь заметным, поскольку серверу все равно придется сформировать полный ResultSet, отсортировать его и только после этого вернуть указанное количество строк:

```
-- Запрос выполнится быстрее  
SELECT TOP 100 * FROM ACCOUNT WHERE ACC_DAT_O < '2014-01-01'  
  
-- Ускорение вряд ли будет ощутимым  
SELECT TOP 100 * FROM ACCOUNT WHERE ACC_DAT_O < '2014-01-01'  
ORDER BY ACC_NAM
```

Ключевое слово **TOP** нельзя использовать в подзапросах.

16. Справочно. Полный синтаксис оператора SELECT.

```
select ::=
    select [top max_number_of_rows] [all | distinct] select_list
    [ into_clause]
    [ from_clause]
    [ where_clause]
    [ group_by_clause]
    [ having_clause]
    [ order_by_clause]
    [ compute_clause]
    [ read_only_clause]
    [ isolation_clause]
    [ browse_clause]
    [ plan_clause]
```

```
select_list ::=
    {*, [alias = ] column_name | column_name [alias] |
    column_name [as alias], expr, @var = expr, ... }
```

```
into_clause ::=
    into [[database.]owner.]table_name
    [ lock {datarows | datapages | allpages }]]
    [ with into_option [, into_option] ...]
into_option ::=
    max_rows_per_page = num_rows
    | exp_row_size = num_bytes
    | reservepagegap = num_pages
    | identity_gap = gap
    [existing table table_name]
    [[external type] at "path_name"
    [column delimiter delimiter]]
```

```
from_clause ::=
    from table_reference [,table_reference ...]
table_reference ::= table_view_name | ANSI_join
table_view_name ::=
    [[database.]owner.] {{table_name | view_name}
    [as] [correlation_name]
    [index {index_name | table_name }}
    [parallel [degree_of_parallelism]]
    [prefetch size ][lru | mru]}
    [holdlock | noholdlock]
    [readpast]
    [shared]
```

```

ANSI_join ::=
    table_reference join_type join table_reference join_conditions
        join_type ::= inner | left [outer] | right [outer]
        join_conditions ::= on search_conditions

where_clause ::=
    where search_conditions

group_by_clause ::=
    group by [all] aggregate_free_expression
    [, aggregate_free_expression]...

having_clause ::=
    having search_conditions

order_by_clause ::=
    order by sort_clause [, sort_clause]...
        sort_clause ::=
            { [[database.]owner.]{table_name.|view_name.}]column_name
              | select_list_number
              | expression }
            [asc | desc]

compute_clause ::=
    compute row_aggregate(column_name) [, row_aggregate(column_name) ...]
    [by column_name [, column_name ...]]

read_only_clause ::=
    for {read only | update [of column_name_list]}

isolation_clause ::=
    at isolation
        { read uncommitted | 0 }
        | { read committed | 1 }
        | { repeatable read | 2 }
        | { serializable | 3 }

browse_clause ::= for browse

plan_clause ::= plan "abstract plan"

```